

MLD — Touche pas au klaxon

Notation : `NOM\_TABLE (clé\_primaire, attribut1, ..., #clé\_étrangère) `

...`

EMPLOYE (id_employe, nom, prenom, email, telephone, mot_de_passe, role)

AGENCE (id_agence, nom_ville)

TRAJET (id_trajet, gdh_depart, gdh_arrivee, nb_places_total, nb_places_dispo,
#id_agence_depart, #id_agence_arrivee, #id_employe)

...`

Clés étrangères de TRAJET

Colonne	Référence	Rôle	
----- ----- -----			
`id_agence_depart`	`AGENCE.id_agence`	agence de départ du trajet	
`id_agence_arrivee`	`AGENCE.id_agence`	agence d'arrivée du trajet	
`id_employe`	`EMPLOYE.id_employe`	auteur du trajet / personne à contacter	

Notes de conception

- Les trois associations du MCD (`PROPOSER`, `DEPART`, `ARRIVER`) sont toutes de cardinalité

`1,1` côté TRAJET et `0,n` côté l'autre entité (EMPLOYE ou AGENCE). Une association `1,n` / `0,n`

se traduit normalement par une table de jonction, mais ici la cardinalité `1,1` permet de porter

la clé étrangère directement dans TRAJET, ce qui est le cas ici (pas de table `PROPOSER`,

`DEPART`, `ARRIVER` séparée).

- `id\_agence\_depart` et `id\_agence\_arrivee` référencent la même table `AGENCE` : il faut donc

deux clés étrangères distinctes (deux `FOREIGN KEY` en SQL) vers `AGENCE.id\_agence`, avec des

noms de colonnes différents pour lever l'ambiguïté.

- `role` dans `EMPLOYE` distingue utilisateur standard / administrateur (ex. valeurs `user`

et `admin`, ou un champ booléen `est\_admin`).

- Une contrainte `CHECK (id\_agence\_depart <> id\_agence\_arrivee)` (ou un contrôle applicatif

équivalent) permettra d'imposer la règle « l'agence de départ et l'agence d'arrivée doivent

être différentes », comme demandé dans le brief.

- `nb\_places\_dispo <= nb\_places\_total` est une autre contrainte de cohérence à vérifier côté

application (et éventuellement en `CHECK` SQL selon le SGBD).